Uppsala University



Regression and Least Squares
Scientific Computing for dataanalysis 2026

Lars Kirchner

Olof Rörby

Contents

1 Introduction **2**

2 Approach and method **2**
2.1 Data processing . . . 2
2.2 Regression . . . 2
2.3 Analysis . . . 3

3 Results **3**
3.1 Plotting of cleaned data . . . 3
3.2 Quadratic fits . . . 3
3.2.1 Comparison and visualization of different quadratic fits . . . 3
3.2.2 Residual analysis . . . 4
3.3 Linear fits . . . 4
3.4 Cubic fits . . . 5
3.5 Condition numbers . . . 6

4 Discussion **6**
4.1 Polynomial fits . . . 6
4.2 Condition numbers . . . 7
4.3 Histograms . . . 7
4.4 Why is it reasonable to avoid higher degree fits? . . . 8

5 References **8**

6 Appendix **8**

A Python Code **8**

1 Introduction

The aim of the project is to determine if is there any significant dependency between life expectancy and schooling. The techniques used are fundamentally analytical in nature but applied to large scale data using python to examine and visualize trends in the data. Due to the nature of problems handling big-data, there is always some degree of error in practical applications. By analyzing how different methods impact the error possible improvement to the original methodology are examined.

2 Approach and method

2.1 Data processing

By importing the whole CSV file into our python workspace we can modify the format and make it more available for analysis. Primarily, data points where the WHO data has a no attribute, meaning no data is present for how long schooling was, need to be excluded from the data to enable numerical analysis. In the datasets there are multiple data points where schooling = 0, this is different to having 'no attribute' and thus cannot be excluded from the analysis - after all, it is wholly possible that someone has received no schooling at all! Plotting our processed data [using python], a rough idea of if there might be any correlation [and causation!] can be obtained.

2.2 Regression

If there is any meaningful causation between schooling and life expectancy, it can be found via analysis of the whole processed dataset. Two different approaches for this analysis are using the so called Normal equations, and QR decomposition. The quadratic regression has the shape of: $y \approx c_0 + c_1 x + c_2 x^2$, from which we can construct the basis for both methods:

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}$$

For the least-squares approach this yields:

$$A \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} \approx \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix}$$

For the normal equations, this then becomes:

$$A^T A \begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = A^T \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix}$$

Using QR decomposition to solve the same problem A is decomposed to: $A = QR$, where Q has orthonormal columns and R is a upper triangular. Using the decomposition: $y = Ac => y \approx QRc <=> Q^TQRc = Q^Ty$, since $Q^TQ = I$ where I is the identity matrix, the expression simplifies to $Rc = Q^Ty$, and since R is a upper triangular, the solution is very manageable.

To find different degrees of polynomials that fit the data, the approach is analogous, where detailed steps can be found in Appendix where the full code lies.

2.3 Analysis

The normality assumption for ordinary least squares regression is that the residuals $r_i = y_i - \hat{y}_i$ follow a normal distribution centered at the mean of zero i.e. $r \sim \mathcal{N}(0, \sigma^2)$. To verify if the normality assumption is applicable to this problem a histogram of the residuals will be plotted using the hist function in python.

3 Results

3.1 Plotting of cleaned data

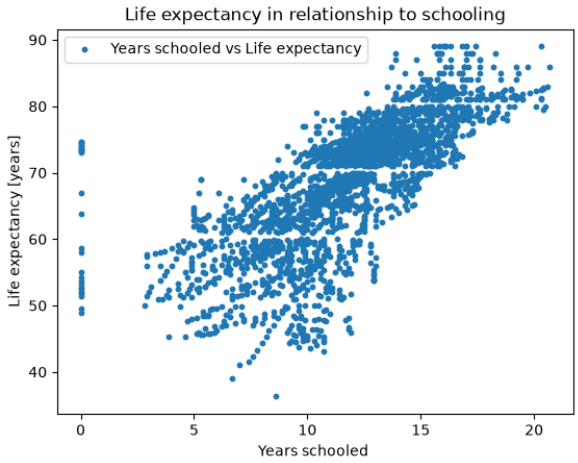


Figure 1: *Scatterplot of raw data. Plotted using python. Full code in appendix.*

3.2 Quadratic fits

For calculations see Python Code in appendix.

3.2.1 Comparison and visualization of different quadratic fits

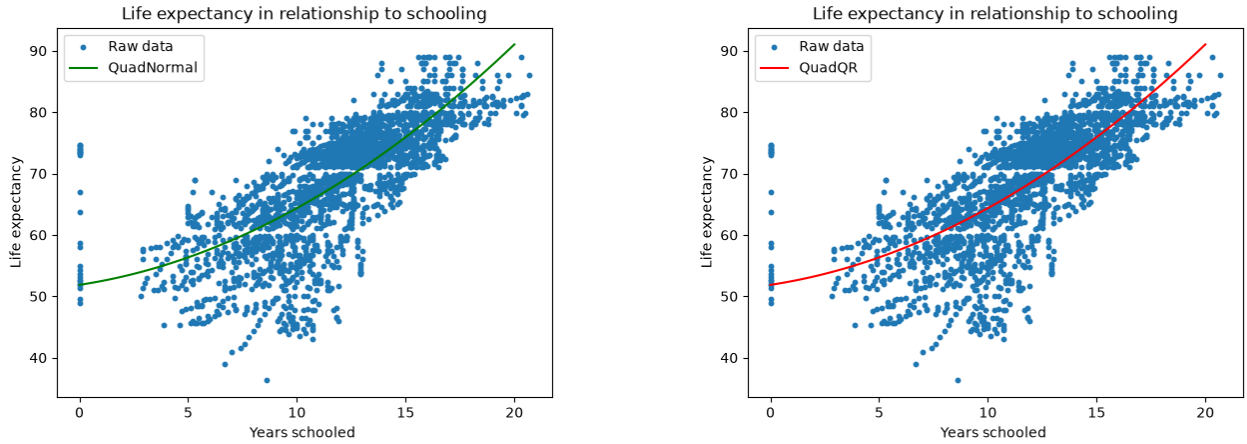
The quadratic fit obtained by using the normal equations [QuadNorm] and the quadratic fit [QuadQR] obtained by using QR decomposition are the same fits:

$$QuadNorm = 51.86257553 + x * 0.54042781 + x^2 * 0.07089593$$

$$QuadQR = 51.86257553 + x * 0.54042781 + x^2 * 0.07089593$$

Primarily, the coefficients are equal considering the number of digits we represent, this is expected since both methods are mathematically sound. However, even though the functions appear

identical, there are factors which impact which method is most useful in a practical application, particularly the condition numbers for each fit [see result and discussion].



(a) Quadratic regression using normal equations. (b) *Quadratic regression using QR decomposition.*

Figure 2: *Comparison of the quadratic regression results.*

3.2.2 Residual analysis

The distribution of residuals was calculated and plotted in python, see Residual analysis in the code.

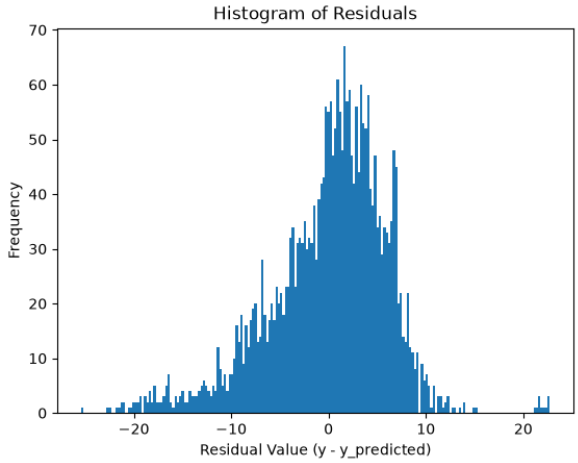
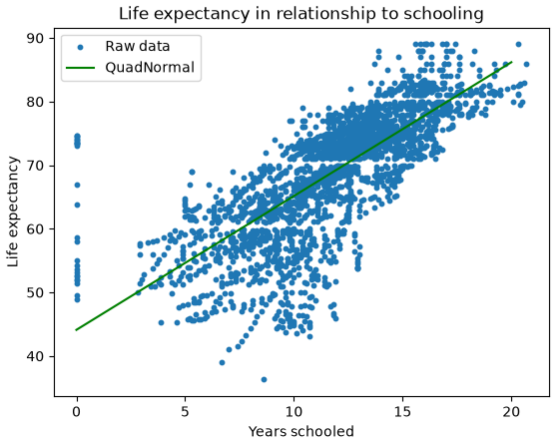


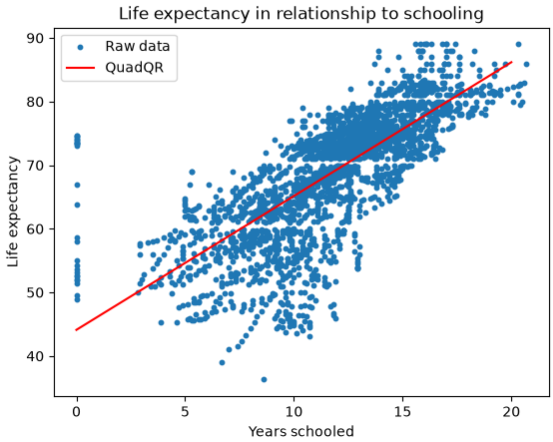
Figure 3: *Histogram of residuals for quadratic fit*

3.3 Linear fits

Using $Pol_degree = 1$ the program plots the linear regression of both Normal equations and QR decomposition, as well as a histogram showing the distribution of residuals:



(a) *Linear regression using normal equations.*



(b) *Linear regression using QR decomposition.*

Figure 4: *Comparison of the linear regression results.*

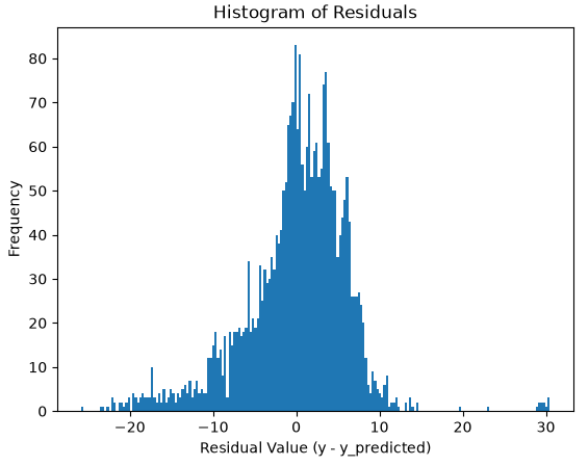
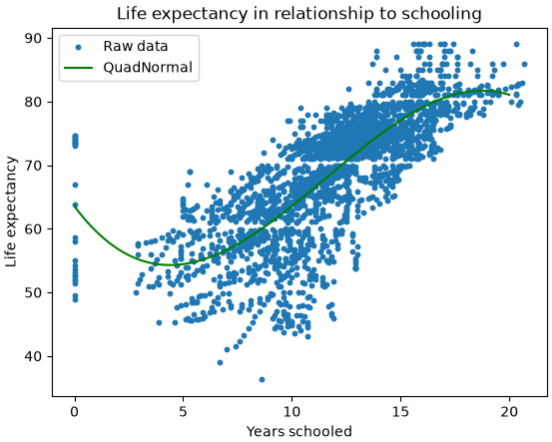


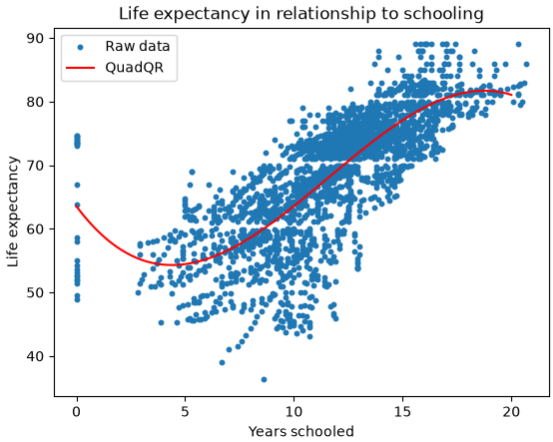
Figure 5: *Histogram of residuals for linear fit*

3.4 Cubic fits

Using $Pol_degree = 3$ the program plots the cubic regression of both Normal equations and QR decomposition, as well as a histogram showing the distribution of residuals:



(a) Cubic regression using normal equations.



(b) *Cubic regression using QR decomposition.*

Figure 6: *Comparison of the Cubic regression results.*

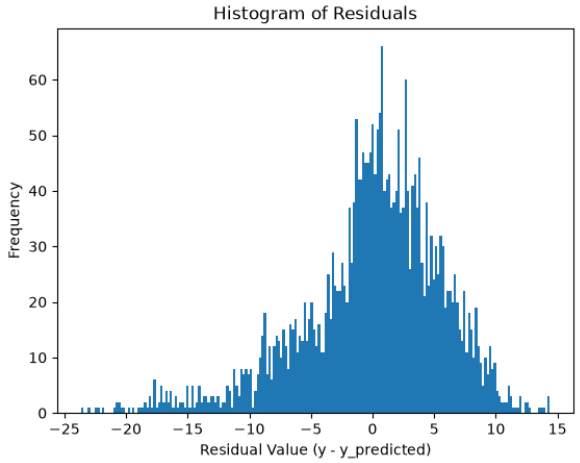


Figure 7: *Histogram of residuals for cubic fit*

3.5 Condition numbers

Table 1: Condition numbers across polynomial fit degrees.

Degree	**Matrix A (QR Approach)**	**Matrix A^TA (Normal Equations)**
1	46.7	2180
2	1270	1.61×10^6
3	25010	6.26×10^8

4 Discussion

4.1 Polynomial fits

As seen in results, both linear fits yield identical coefficients, however, the condition numbers for the approaches different greatly, making QR decomposition the clear choice since the calculations

for both methods can be seen as equally taxing. Between calculations of different polynomial degrees using python there is no meaningful added effort, however it likely takes slightly more compute for higher degrees. To determine which degree a given fit should have, the condition number plays a large role.

4.2 Condition numbers

When solving linear equation systems, the condition number is a measure of how sensitive the system is to disturbances in the input. The condition number represents the maximal amplification of the input error, and does not guarantee that the error is amplified by the value of the condition number. Ideally, the condition number close to 1. As seen in Results, the magnitude of the condition numbers for fits calculated using the normal equations explode, making the method ill-suited for practical applications. Using QR decomposition, the condition number increases more slowly, making the approach more suited for applications.

By taking $\log_{10}(\kappa)$, where κ is the condition number of the design matrix A or A^TA, we obtain an estimate for the worst-case number of lost digits of precision:

$$\begin{aligned} \text{QR Decomposition: } & \log_{10}(2.5012 \times 10^4) \approx 4.4 \text{ digits lost} \\ \text{Normal Equations: } & \log_{10}(6.2560 \times 10^8) \approx 8.8 \text{ digits lost} \end{aligned}$$

Treating the QR solution x_{QR} as the exact baseline, the actual error in the Normal Equations solution x_{NE} yields:

$$\begin{aligned} \text{Absolute Error: } & \|x_{NE} - x_{QR}\|_2 \approx 9.6419 \times 10^{-12} \\ \text{Relative Error: } & \frac{\|x_{NE} - x_{QR}\|_2}{\|x_{QR}\|_2} \approx 1.51 \times 10^{-13} \end{aligned}$$

Comparing this relative error of 1.51×10^{-13} to machine precision ($\epsilon_{\text{mach}} \approx 10^{-16}$) shows an actual loss of roughly 3 decimal digits. This highlights that the condition number provides a pessimistic upper bound on error amplification, whereas the actual observed error in this specific instance is significantly lower.

By comparing these errors to machine epsilon, roughly 10^{-16}, the error is significant. However, if compared to the very likely rounding of time in school [likely a decimal average] to integers in the WHO dataset, this error should be negligible.

As we can see for both cubic fits, the fit data does not seem to match the data that well for small values of schooling. It also seems unlikely that life expectancy would have a global minimum at roughly five years of schooling. As mentioned in Approach in method, this discrepancy stems from the fact that some values in the raw data are zero, which skews the fits for smaller schooling values. The data-points where schooling = 0 are clear visual outliers, and one might want to exclude these points if , for example, the aim is to make a model which has high accuracy at longer schooling periods. A way to resolve this would be to try to determine if the schooling = 0 values are in some way invalid, for example if the country has reported zero instead of None [i.e. no data], since it seems unreasonable that the average schooling for a country in a given year would be zero.

4.3 Histograms

The histograms in fig.3, fig.5 and fig.7 all follow a rough pattern of a normal distribution. If the histogram for the quadratic fit in fig.3 is considered the baseline, it looks as though the distribution

is centered around zero. Comparing the distributions for both linear and cubic fit in fig.5 and fig.7 to the baseline shows that they too are centered around zero. All the histograms are skewed to the negative left side. This is probably a result of the outlier data points with a relatively low life expectancy compared to the years schooled, or of the data points where schooling = 0. However, the outliers in the positive ends of the distributions are a bit different. It seems that a higher polynomial degree makes the extreme positive residuals tighter (from +30 in linear to +15 in cubic), resulting in a bit more slope shape from the peak on the right hand side.

4.4 Why is it reasonable to avoid higher degree fits?

As seen for the conditions numbers for linear, quadratic and cubic fit, the values explode very quickly which means that in the worst case, the error present can be magnified by said value. For the normal equations, A is squared, causing its value to increase incredibly fast. Using QR decomposition, the condition number increases more slowly, since the condition number of A i roughly to that of R, and thus no square is applied.

For high degree polynomials fitted to data, erratic oscillations become increasingly frequent as the degree increases, causing instability within the model. This decreases the usefulness of the model, especially for practical applications.

5 References

1. Instructions to Miniproject 1 within the course Scientific computing for data analysis, 2026, department of Information technology, Uppsala University.
2. WHO data on life expectancy as and schooling [among other parameters]. Link: LifeExpectancyData.csv.

6 Appendix

A Python Code

The following appendix contains the complete Python implementation used for data cleaning, quadratic regression, and visualization. Both the normal-equation approach and the QR-decomposition approach are included. (Note that after the improved versions in part 2, the funcitons are still called *quad_..* simply because of how the code was written in part 1 with the main function etc. even though its confusing!)

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.polynomial import Polynomial
import pandas as pd

### Handle data ###
file_path = r"C:\Users\lasse\Downloads\BerVet2_lokal\MiniProjects\
   Regression and least squares\LifeExpectancyData.csv"

data = pd.read_csv(file_path)

data.columns = data.columns.str.strip()
```

```
12
13 # Keep only the two columns
14 data = data[["Life expectancy", "Schooling"]]
15
16 data["Life expectancy"] = pd.to_numeric(
17     data["Life expectancy"], errors="coerce"
18 )
19
20 data["Schooling"] = pd.to_numeric(
21     data["Schooling"], errors="coerce"
22 )
23
24 cleaned_data = data[~np.isnan(data).any(axis=1)]
25
26 x = cleaned_data["Schooling"].to_numpy()
27 y = cleaned_data["Life expectancy"].to_numpy()
28
29
30 ### Test cleaned data ###
31 def test_cleaned_data():
32     tests_ok = 0
33
34     if np.isnan(x).any() == False:
35         tests_ok += 1
36
37     if np.isnan(y).any() == False:
38         tests_ok += 1
39
40     if np.isfinite(x).all() == True:
41         tests_ok += 1
42
43     if np.isfinite(y).all() == True:
44         tests_ok += 1
45
46     if tests_ok == 4:
47         return True
48     else:
49         return False
50
51
52 ### Plot cleaned data ###
53 def plot_cleaned_data():
54
55     plt.scatter(x, y, s=10)
56
57     plt.title(
58         "Life expectancy in relationship to schooling"
59     )
60
61     plt.xlabel("Years schooled")
62     plt.ylabel("Life expectancy [years]")
63
```

```
    plt.legend(
        ["Years schooled vs Life expectancy"]
    )

    plt.show()
    plt.figure(1)


"""### Quadratic polynomial ###
A = np.column_stack((
    np.ones(len(x)),
    x,
    x**2
))"""

### improved version###

Pol_degree = 3

A_columns = [np.ones(len(x))]
for deg in range(1, Pol_degree + 1):
    A_columns.append(x**deg)

A = np.column_stack(A_columns)


### Normal equations ###
ATA = A.T @ A
ATy = A.T @ y

coefficients_normal = np.linalg.solve(
    ATA,
    ATy
)


"""def quad_normal(x):

    q2_norm = Polynomial([
        coefficients_normal[0],
        coefficients_normal[1],
        coefficients_normal[2]
    ])

    return q2_norm(x)"""

### improved version without hardlock ###

def quad_normal(x):
    q2_norm = Polynomial(coefficients_normal)
    return q2_norm(x)

```

```
### QR decomposition ###
Q, R = np.linalg.qr(A)

coefficients_qr = np.linalg.solve(
    R,
    Q.T @ y
)


"""def quad_qr(x):

    q2_qr = Polynomial([
        coefficients_qr[0],
        coefficients_qr[1],
        coefficients_qr[2]
    ])

    return q2_qr(x)"""

### improved version without hardlock ###

def quad_qr(x):
    q2_qr = Polynomial(coefficients_qr)
    return q2_qr(x)


### Plot quadratic fit using normal equations ###
def plot_quad_normal():

    plt.scatter(
        x,
        y,
        s=10,
        label="Raw data"
    )

    years_schooled = np.linspace(0, 20, 100)

    plt.plot(
        years_schooled,
        quad_normal(years_schooled),
        label="QuadNormal",
        color="green"
    )

    plt.title(
        "Life expectancy in relationship to schooling"
    )

    plt.xlabel("Years schooled")
    plt.ylabel("Life expectancy")

```

```
    plt.legend()
    plt.show()
    plt.figure(2)


### Plot quadratic fit using QR decomposition ###
def plot_quad_qr():

    plt.scatter(
        x,
        y,
        s=10,
        label="Raw data"
    )

    years_schooled = np.linspace(0, 20, 100)

    plt.plot(
        years_schooled,
        quad_qr(years_schooled),
        label="QuadQR",
        color="red"
    )

    plt.title(
        "Life expectancy in relationship to schooling"
    )

    plt.xlabel("Years schooled")
    plt.ylabel("Life expectancy")

    plt.legend()
    plt.show()
    plt.figure(3)

### Residual Analysis ###
def plot_residuals():
    y_predicted = quad_normal(x)
    residuals = y - y_predicted

    # Plot histogram of residuals
    plt.hist(residuals, bins=200)

    plt.title("Histogram of Residuals")
    plt.xlabel("Residual Value (y - y_predicted)")
    plt.ylabel("Frequency")

    plt.show()
    plt.figure(4)

### Condition Numbers ###
cond_A = np.linalg.cond(A)
```

```
cond_ATA = np.linalg.cond(ATA)

### Main function ###
def main():

    def _test_cleaned_data():

        if test_cleaned_data() == True:
            print("Cleaning of data ok.")
        else:
            print("Cleaning of data failed")

    _test_cleaned_data()

    print(
        "Visualize raw data by plotting cleaned data."
    )

    plot_cleaned_data()

    ### Comparing QR vs normal equations ###
    print(
        "Comparing the resulting quadratic functions "
        "from the normal equations and QR respectively."
    )

    print(
        f"The coefficients for the normal equation "
        f"approach are: {coefficients_normal}\n"
    )

    print(
        f"The coefficients for the QR approach are: "
        f"{coefficients_qr}\n"
    )

    print(
        "Visualize raw data alongside different "
        "quadratic fits."
    )

    plot_quad_normal()
    plot_quad_qr()
    plot_residuals()


    print(f"Condition number of matrix A (QR approach): {cond_A}")
    print(f"Condition number of matrix ATA (Normal equations): {cond_ATA}
   ")

    def condition_number_calc():
```

```
        print('\n Verify that the condition numbers are correctly calculated. ')
        print(f'625599929.8927681/25011.995720560757 = {625599929.8927681/25011.995720560757}')
        print(f'25011.995720560757^2 = {25011.995720560757**2}')

    condition_number_calc()

    def precision_loss_estimate():
        ### Condition Numbers
        cond_A = np.linalg.cond(A)
        cond_ATA = np.linalg.cond(ATA)

        # Precision loss estimatins
        digits_lost_qr = np.log10(cond_A)
        digits_remaining_qr = max(0, 16 - digits_lost_qr)

        digits_lost_normal = np.log10(cond_ATA)
        digits_remaining_normal = max(0, 16 - digits_lost_normal)

        # detials
        print("       NUMERICAL PRECISION & ACCURACY ANALYSIS       ")
        print(f"Polynomial Degree Used: {Pol_degree}")

        print("NORMAL EQUATIONS APPROACH (A^T * A):")
        print(f"  Coefficients:     {coefficients_normal}")
        print(f"  Condition Number: {cond_ATA:.4e}")

        print("QR DECOMPOSITION APPROACH (Q * R):")
        print(f"  Coefficients:     {coefficients_qr}")
        print(f"  Condition Number: {cond_A:.4e}")

        # Relative difference between the two coefficient solutions
        rel_diff = np.linalg.norm(coefficients_normal - coefficients_qr) / np.linalg.norm(coefficients_qr)
        print(f"Relative difference: {rel_diff}")

    precision_loss_estimate()

main()
```

Listing 1: *Python implementation of the regression analysis*